

ExamSense+

Documentație completă, explicată pe înțelesul tuturor

Scopul acestui document este să explice tot proiectul: ce face aplicația, cum circulă datele, ce tehnologii folosește, cum este securizată și ce trebuie să poți explica unui profesor.

Versiune orientată spre înțelegere practică, nu spre limbaj academic.

Aplicație	ExamSense+
Tip proiect	Aplicație web full-stack pentru învățare și autoevaluare
Stack principal	React + Vite + Axios + Tailwind / Django + DRF + SQLite
Flux principal	Upload document -> extragere concepte/definiții -> generare quiz -> rezolvare -> statistici și recomandări

1. Imaginea de ansamblu

ExamSense+ este o platformă web care pornește de la un material teoretic - de obicei un PDF sau un DOCX - și îl transformă într-un instrument de învățare. Utilizatorul nu primește doar o listă de fișiere, ci un flux complet: încarcă documentul, sistemul extrage ideile importante, generează întrebări, apoi salvează rezultatele și arată progresul.

Cu alte cuvinte, proiectul tău nu este doar un site cu autentificare și upload. Este o aplicație care leagă împreună trei lucruri importante: gestionarea utilizatorilor, procesarea documentelor și evaluarea automată.

1.1. Ce problemă rezolvă

- Mulți studenți au materiale lungi, greu de parcurs și fără instrumente rapide de verificare a înțelegerii.
- În mod normal, transformarea unui curs în seturi de întrebări se face manual și consumă timp.
- Aplicația reduce acest efort: dintr-un document brut obții definiții extrase, întrebări și istoric de progres.

1.2. Ce poate face utilizatorul

- să își creeze cont și să se autentifice
- să își gestioneze profilul și parola
- să încarce documente PDF și DOCX
- să vizualizeze definițiile extrase din fiecare document
- să rezolve quiz-uri generate automat
- să refacă același quiz sau să genereze un quiz nou
- să vadă istoricul încercărilor
- să vadă dashboard cu statistici, concepte slabe și recomandări

2. Arhitectura aplicației

Proiectul este full-stack. Asta înseamnă că are o parte de interfață (frontend), o parte de logică și date (backend) și o bază de date care stochează starea aplicației.

Legătura dintre aceste părți poate fi explicată simplu prin fluxul de mai jos:

```
Browser -> Frontend React (Vite, JavaScript, Tailwind)
Frontend -> Axios -> request HTTP
Request -> Backend Django REST Framework
Backend -> modele Django -> baza de date SQLite
Backend -> răspuns JSON
JSON -> Axios -> React actualizează interfața
```

2.1. Rolul fiecărei părți

Componentă	Ce este	Rol în proiect
Frontend	Interfața văzută de utilizator	Afișează pagini, formulare, dashboard, documente, quiz-uri
Backend	Serverul aplicației	Validează date, procesează documente, generează întrebări, răspunde la requesturi
Baza de date	Stocarea persistentă	Păstrează utilizatori, documente, definiții, întrebări, attempturi și răspunsuri

Browserul	Clientul final	Rulează React și trimite requesturi către backend
-----------	----------------	---

3. Frontendul: ce folosești și la ce

Frontendul este scris în JavaScript, folosind React și Vite. El nu conține logica grea de business și nu lucrează direct cu baza de date. Rolul lui este să afișeze datele, să preia acțiunile utilizatorului și să comunice cu backendul.

3.1. React

React este biblioteca principală pentru interfață. Ai împărțit UI-ul în componente și pagini. Asta înseamnă că în loc să ai un fișier uriaș cu tot site-ul, ai bucăți mai mici, reutilizabile.

- componente comune: SectionCard, StatCard, ErrorAlert, ConfirmDialog, QuizOptionsDialog
- componente de layout: Navbar, MobileMenu, ProtectedRoute, ProtectedAdminRoute
- pagini: LoginPage, RegisterPage, ProfilePage, UploadPage, DocumentsPage, DocumentDetailPage, QuizPlayPage, DashboardPage etc.

3.2. JavaScript

JavaScript este limbajul cu care ai scris logica frontendului: schimbarea stării componentelor, tratarea formularelor, validări, navigare între pagini și afișarea rezultatelor primite de la backend.

3.3. Vite și rolul lui Node.js

Vite este tool-ul cu care rulezi și construiești frontendul. Când pornești proiectul cu comenzi de tip npm run dev, de fapt folosești ecosistemul Node.js pentru partea de dezvoltare a frontendului.

Important: Node.js nu este serverul final al aplicației tale în arhitectura actuală. El este folosit în dezvoltare pentru a porni Vite și pentru a construi build-ul frontendului. Backendul real rămâne Django.

- Node.js = runtime pentru ecosistemul frontend
- npm = managerul de pachete și scripturi
- Vite = server de dezvoltare rapid + build tool pentru React

3.4. Axios

Axios este clientul HTTP din frontend. El este folosit pentru a trimite requesturi către backend și pentru a primi răspunsuri, de obicei în format JSON. Nu face același lucru ca Django REST Framework: DRF expune API-ul pe backend, Axios îl consumă pe frontend.

```
GET /auth/me/ -> citește utilizatorul autentificat
POST /auth/login/ -> trimite datele de autentificare
POST /documents/upload/ -> trimite fișierul încărcat
GET /documents/ -> ia lista documentelor
POST /documents/submit-quiz/ -> trimite răspunsurile quiz-ului
GET /learning/dashboard/ -> ia datele pentru grafice și statistici
```

3.5. Tailwind CSS

Tailwind este folosit pentru stilizare. În loc să scrii mult CSS separat, ai clase direct în componente, pentru spațiere, culori, layout, responsive design și efecte vizuale.

3.6. Alte librării din frontend

Bibliotecă	Rol
react-router-dom	navigare între pagini și rute protejate
framer-motion	animații simple și tranziții mai plăcute
lucide-react	iconițe
recharts	grafice pentru dashboard

4. Backendul: ce folosești și la ce

Backendul este scris în Python, folosind Django și Django REST Framework. Aici trăiește logica reală a aplicației: autentificarea, accesul la baza de date, procesarea documentelor, generarea întrebărilor și calculele pentru progres.

4.1. Python

Python este limbajul de programare folosit în backend. Toate funcțiile, clasele și regulile tale sunt scrise în Python: extragerea textului, procesarea definițiilor, validările, generarea întrebărilor și endpoint-urile.

4.2. Django

Django este frameworkul web construit peste Python. El îți oferă structura aplicației: settings, modele, rute, middleware, autentificare, admin și integrarea cu baza de date.

4.3. Django REST Framework

Django REST Framework este extensia folosită pentru a crea API-uri. El este motivul pentru care backendul tău nu trimite HTML, ci JSON. Cu DRF ai construit APIView-uri, serializare, permisiuni și răspunsuri API.

Diferența simplă este: Django simplu generează pagini web, DRF generează API-uri.

4.4. Organizarea pe aplicații

App	Rol
users	autentificare, profil, schimbare/resetare parolă, throttles și autentificare custom cu cookies
documents	documente, definiții, întrebări, attempturi, răspunsuri, upload și submit quiz
learning	dashboard, concepte slabe, recomandări, retry quiz
adminpanel	panou de administrare separat de Django admin
nlp	curățarea textului, detectarea limbii, extragerea definițiilor
quiz	logica de generare a întrebărilor

5. Baza de date: ce salvezi și unde

În versiunea actuală locală a proiectului, datele sunt salvate în SQLite. Asta înseamnă că baza de date este un singur fișier, de regulă db.sqlite3, pe care Django îl folosește direct.

5.1. Ce este SQLite

SQLite este o bază de date foarte bună pentru dezvoltare locală și proiecte mici sau medii. Avantajul ei este simplitatea: nu trebuie să pornești un server separat de bază de date. Dezavantajul principal este că nu este cea mai bună alegere pentru producție publică cu trafic mai mare.

5.2. Ce date se salvează

- utilizatorii și datele lor de profil
- documentele încărcate de utilizatori
- textul extras din documente
- definițiile extrase
- întrebările generate
- încercările de quiz (attempturi)
- răspunsurile date de utilizatori

5.3. Modelele importante

Model	Ce reprezintă	Exemple de câmpuri
User	contul utilizatorului	username, email, first_name, last_name, is_staff
Document	un fișier încărcat	user, file, extracted_text, uploaded_at
ExtractedDefinition	o definiție extrasă	concept, definition, pattern, language, sentence
GeneratedQuestion	o întrebare de quiz	question_type, question_text, options, correct_answer
QuizAttempt	o sesiune completă de quiz	user, document, score, total_questions, completed_at
QuizAnswer	răspunsul la o întrebare	attempt, question, selected_answer, is_correct

Un lucru important de explicat profesorului este că utilizatorii nu sunt salvați în frontend și nici în cookies ca sursă de adevăr. Ei sunt salvați în baza de date prin modelul Django de utilizator, iar parolele sunt stocate hash-uit, nu în clar.

6. Workflow-ul complet al aplicației

6.1. Register și login

Fluxul începe prin crearea unui cont sau autentificare. Frontendul trimite formularul către backend. Backendul validează datele, creează utilizatorul sau verifică parola, apoi răspunde cu datele necesare.

La login, backendul generează tokenuri JWT și le pune în cookies. Frontendul nu ține parola și nu decide singur dacă utilizatorul este autentificat; el întreabă backendul prin endpoint-ul /auth/me/.

6.2. Încărcarea unui document

Utilizatorul intră în UploadPage și alege fișierul, dificultatea și numărul de întrebări. Frontendul validează local că fișierul există, că are extensie acceptată și că numărul de întrebări este într-un interval valid.

Apoi frontendul folosește FormData și trimite fișierul la backend prin Axios.

6.3. Ce face backendul după upload

1. verifică dacă fișierul există
2. validează extensia și content type-ul
3. verifică dimensiunea maximă permisă
4. salvează documentul în modelul Document
5. extrage textul din PDF sau DOCX
6. salvează textul extras
7. procesează textul pentru a găsi definiții
8. generează întrebări pe baza definițiilor
9. trimite înapoi documentul procesat și datele asociate

6.4. Extragerea textului

Pentru PDF folosești PyPDF2, iar pentru DOCX folosești python-docx. Aceste biblioteci citesc conținutul textual al fișierului și îl transformă într-un text brut pe care îl poate procesa pipeline-ul NLP.

6.5. Extragerea definițiilor și conceptelor

Aici este una dintre părțile cele mai importante ale proiectului. Pipeline-ul tău NLP nu este bazat pe un model AI extern, ci pe reguli scrise de tine și pe biblioteci auxiliare.

10. se curăță textul
11. se detectează limba (română sau engleză)
12. textul este împărțit în propoziții
13. se caută pattern-uri care semnalează definiții, cum ar fi este, reprezintă, is, means
14. se curăță conceptul extras
15. se filtrează definițiile și conceptele slabe
16. se sortează și se elimină duplicatele

6.6. Generarea întrebărilor

După ce ai definițiile extrase, sistemul construiește întrebări. Ai implementat mai multe tipuri: mcq, true_false și mcq_reverse. Generatorul alege formulări diferite în funcție de limba definiției și de intenția detectată: definiție generală, rol, utilizare, structură, proces sau referință.

6.7. Rezolvarea quiz-ului

Frontendul afișează toate întrebările, permite navigarea între ele, salvează răspunsurile în state și trimite totul la final către backend. Backendul calculează scorul, creează un QuizAttempt, creează câte un QuizAnswer pentru fiecare întrebare și răspunde cu rezultatul complet.

6.8. Dashboard, istoricul și recomandările

După acumularea attempturilor, sistemul poate calcula statistici: scor mediu, scor minim și maxim, câte răspunsuri sunt corecte sau greșite, care concepte sunt slabe și ce întrebări se potrivesc pentru recapitulare.

7. Cum circulă datele între frontend, backend și server

Este important să poți explica traseul unei acțiuni simple. Exemplu: utilizatorul apasă pe Login.

17. Browserul încarcă frontendul React.
18. În LoginPage, utilizatorul completează username și parolă.

19. React salvează valorile în state și la submit apelează funcția de login.
20. Axios trimite un request POST către /auth/login/.
21. Django REST Framework primește requestul, validează datele și autentifică utilizatorul.
22. Backendul trimite răspuns JSON și setează cookies cu tokenurile necesare.
23. Frontendul afișează toast de succes și navighează către /documents.

7.1. Ce este un request și ce este un response

Un request este cererea trimisă de frontend către backend. Un response este răspunsul serverului. La tine, aceste răspunsuri sunt în general JSON.

7.2. Tipurile principale de requesturi folosite

Metodă HTTP	Când o folosești	Exemple
GET	când citești date	/documents/, /auth/me/, /learning/dashboard/
POST	când creezi sau trimiți ceva nou	/auth/login/, /documents/upload/, /documents/submit-quiz/
PUT	când actualizezi complet o resursă	/auth/profile/
PATCH	când actualizezi parțial	toggle active în admin panel
DELETE	când ștergi o resursă	/documents/<id>/delete/

8. Autentificare și securitate

8.1. JWT

JWT înseamnă JSON Web Token. La tine este mecanismul folosit pentru a reprezenta sesiunea utilizatorului după login. Backendul emite tokenuri și le verifică la requesturile ulterioare.

8.2. Cookies

Tokenurile sunt păstrate în cookies, nu în localStorage ca mecanism principal. Asta este potrivit pentru o aplicație web, pentru că browserul trimite automat cookie-urile la requesturi către backend.

8.3. CSRF

Pentru că folosești cookies, trebuie să protejezi aplicația împotriva atacurilor CSRF. Backendul setează un cookie CSRF, iar frontendul îl citește și îl trimite în header prin Axios. Backendul verifică apoi acest token.

8.4. CORS

Frontendul și backendul rulează pe origini diferite în dezvoltare. Browserul nu permite automat astfel de requesturi, deci backendul trebuie să spună explicit ce origini are voie să accepte. Aici intervine configurarea CORS.

8.5. Setări de securitate folosite sau pregătite

- filtre și headere de securitate pentru browser
- X-Frame-Options pentru a evita încărcarea aplicației în iframe-uri nedorite
- nosniff pentru tipuri de conținut
- configurații secure pentru cookies în producție
- protecție la parole prin hashing Django

- permisiuni separate pentru utilizatori normali și admini

8.6. Throttling și limite

Ai și throttles pentru anumite acțiuni sensibile, cum ar fi login, register sau submit quiz. Ideea este să reduci abuzul și flood-ul de requesturi.

9. Validări și controlul erorilor

Proiectul validează datele în două locuri: în frontend și în backend.

9.1. Frontend

- validează email, username, parole și fișiere înainte de submit
- afișează erori live doar când câmpurile au fost atinse
- folosește toasts și ErrorAlert pentru feedback prietenos

9.2. Backend

- verifică din nou aceleași condiții esențiale
- respinge fișiere prea mari sau de tip greșit
- verifică permisiunile pe documente și attempturi
- validează parolele și unicitatea username/email

Este important de explicat că backendul este sursa finală de adevăr. Chiar dacă frontendul validează, backendul trebuie să valideze din nou, pentru că orice request poate fi falsificat.

10. Ce înseamnă fiecare termen important

Termen	Explicație
Python	limbajul de programare folosit pentru backend
Django	frameworkul web principal al backendului
Django REST Framework	stratul de API peste Django
JavaScript	limbajul folosit în frontend
React	biblioteca pentru construirea interfeței
Vite	tool de dezvoltare și build pentru frontend
Node.js	runtime-ul folosit în ecosistemul frontend, mai ales pentru Vite și npm
Axios	clientul HTTP din frontend
SQLite	baza de date locală actuală a proiectului
PostgreSQL	bază de date recomandată pentru producție, nu cea folosită acum local
JSON	formatul de date trimis între frontend și backend
JWT	token de autentificare
Cookie	mecanism de stocare trimis de browser către server
CORS	regulile care permit comunicarea între origini diferite
CSRF	protecție împotriva requesturilor malițioase făcute în numele utilizatorului
WSGI	interfața clasică de rulare pentru aplicații Django
ASGI	interfața modernă pentru aplicații asincrone și

11. WSGI și ASGI în proiect

Proiectul tău include atât `wsgi.py`, cât și `asgi.py`. Asta este normal într-un proiect Django modern.

- WSGI este standardul clasic pentru aplicații web Python și este suficient pentru un backend Django obișnuit.
- ASGI este standardul modern, util când vrei suport mai bun pentru lucruri asincrone, WebSockets sau scalare diferită.
- În proiectul actual, aplicația este în principal request/response clasică, deci WSGI este suficient pentru rularea standard.

12. Faviconuri, iconițe și identitatea vizuală

În interfață folosești iconițe din `lucide-react`. Acestea nu sunt același lucru cu faviconul browserului. Iconițele apar în butoane, meniuri și carduri.

Faviconul este iconița mică din tab-ul browserului. În proiecte de tip React/Vite, faviconul este de regulă un asset pus în folderul public sau configurat în fișierele de bază HTML. Ca explicație pentru profesor, este suficient să spui că faviconul și identitatea vizuală sunt tratate ca asset-uri frontend, separate de logica aplicației.

Dacă vrei o trasabilitate perfectă, sursa exactă a fișierului favicon se verifică în structura de `assets/public` a proiectului.

13. Cum este organizată interfața pentru utilizator

- `HomePage` - explică scopul aplicației
- `Login/Register` - intrare în sistem
- `DocumentsPage` - lista documentelor utilizatorului
- `UploadPage` - încărcare și procesare document nou
- `DocumentDetailPage` - definiții și întrebări generate pentru un document
- `QuizPlayPage` - rezolvarea quiz-ului
- `QuizResultPage` - rezultatul ultimei rezolvări
- `QuizHistoryPage` și `QuizAttemptDetailPage` - istoricul și detaliile complete
- `DashboardPage` - statistici, grafice și puncte slabe
- `RecommendationsPage` și `WeakConceptsPage` - recomandări și recapitulare

14. Cum funcționează panoul de admin

Ai implementat un admin panel separat în aplicație, diferit de Django admin. Asta este o decizie bună, pentru că îți permite să controlezi exact ce vede și ce poate face un administrator în interfața principală.

- adminul vede statistici globale
- poate vedea utilizatorii existenți
- poate activa sau dezactiva utilizatori
- poate vedea documente încărcate de toți utilizatorii
- poate șterge documente
- poate vedea attempturile globale

15. Ce ai putea spune clar unui profesor

O formulare bună, scurtă și clară ar fi următoarea:

Am realizat o aplicație web full-stack numită ExamSense+, care transformă documente de curs în quiz-uri și statistici de progres. Frontendul este construit în React, JavaScript și Tailwind, iar backendul în Python cu Django și Django REST Framework. Utilizatorii își creează cont, încarcă documente PDF sau DOCX, iar backendul extrage textul, identifică definiții și generează întrebări automat. Rezultatele la quiz-uri sunt salvate în baza de date SQLite, iar aplicația oferă istoric, dashboard, concepte slabe și recomandări. Am implementat și autentificare cu JWT în cookies, protecție CORS și CSRF, limitări de upload și un panou de administrare.

15.1. Cum să descrii contribuția ta

- am construit arhitectura full-stack a aplicației
- am realizat frontendul React și componentizarea interfeței
- am implementat API-urile în Django REST Framework
- am creat modelele bazei de date
- am implementat fluxul de upload și procesare documente
- am scris pipeline-ul NLP bazat pe reguli pentru extragerea definițiilor
- am implementat generarea întrebărilor și logica de quiz
- am construit dashboardul, istoricul și recomandările
- am adăugat autentificare, securitate și panou de admin

16. Întrebări frecvente și răspunsuri scurte

24. **Ce este aplicația?** O platformă care transformă documente de studiu în quiz-uri și statistici de progres.
25. **De ce ai ales React?** Pentru interfață modulară, interactivă și ușor de extins.
26. **De ce ai ales Django?** Pentru că oferă rapid structură de backend, autentificare, modele și securitate.
27. **DRF și Axios fac același lucru?** Nu. DRF expune API-ul pe backend, Axios îl consumă pe frontend.
28. **Unde se salvează utilizatorii?** În baza de date SQLite, prin modelul de utilizator Django.
29. **Unde se salvează documentele?** Metadatele în baza de date; fișierele în storage-ul local configurat pentru media.
30. **Folosești PostgreSQL acum?** Nu în versiunea locală actuală; folosesc SQLite. PostgreSQL este pasul natural pentru producție.
31. **Ce face JWT?** Reprezintă sesiunea utilizatorului după login.
32. **De ce folosești cookies?** Pentru un flux web natural și mai sigur decât expunerea tokenurilor direct în frontend.
33. **De ce ai nevoie și de CSRF dacă ai JWT?** Pentru că tokenurile sunt în cookies și browserul le trimite automat; CSRF protejează requesturile sensibile.
34. **Ce înseamnă upload limit?** Limită de mărime și tip pentru fișierele încărcate, pentru securitate și stabilitate.
35. **Ce sunt WSGI și ASGI?** Moduri standard prin care aplicația Django poate fi rulată pe server.

17. Ce ar trebui să știi foarte bine la final

- Aplicația este full-stack: React în frontend, Django în backend.
- Frontendul nu vorbește direct cu baza de date; vorbește cu backendul prin Axios și API-uri JSON.
- Backendul folosește Django REST Framework pentru endpoint-uri și SQLite pentru stocarea locală a datelor.

- Documentele încărcate sunt procesate în backend: se extrage textul, se caută definiții, se generează întrebări.
- Autentificarea folosește JWT în cookies, protejate prin mecanisme precum CSRF și configurări de securitate.
- Dashboardul și recomandările sunt construite din datele salvate în attempturi și răspunsuri.
- Node.js și Vite sunt folosite pentru ecosistemul frontend, nu ca server principal al aplicației.
- PostgreSQL este relevant mai ales ca pas viitor pentru publicare, nu ca tehnologie folosită acum în varianta locală.

Dacă stăpânești clar toate punctele de mai sus, poți răspunde bine la majoritatea întrebărilor despre proiect.

18. Posibile îmbunătățiri viitoare

- introducerea unui mod AI de extragere concepte și generare întrebări, comparabil cu modul NLP
- salvarea metodei de generare pentru comparații de calitate
- migrare de la SQLite la PostgreSQL pentru producție
- deploy public cu frontend și backend separate
- storage extern pentru fișiere media
- feedback explicit de la utilizator pentru calitatea întrebărilor

Important: aceste îmbunătățiri viitoare nu anulează valoarea versiunii actuale. Versiunea actuală este deja un proiect complet, funcțional și explicabil.

19. Concluzie finală

ExamSense+ este o aplicație web completă, bine structurată, care îmbină managementul utilizatorilor, procesarea documentelor și evaluarea automată. Nu este doar un CRUD cu autentificare și nici doar un script de procesare text. Este un sistem coerent, în care fiecare parte - frontend, backend, bază de date, securitate, quiz-uri și statistici - are un rol clar.

Cel mai bun mod de a o prezenta este acesta: pornește de la un document brut și îl transformă într-o experiență interactivă de învățare, păstrând în același timp istoricul și analiza progresului.